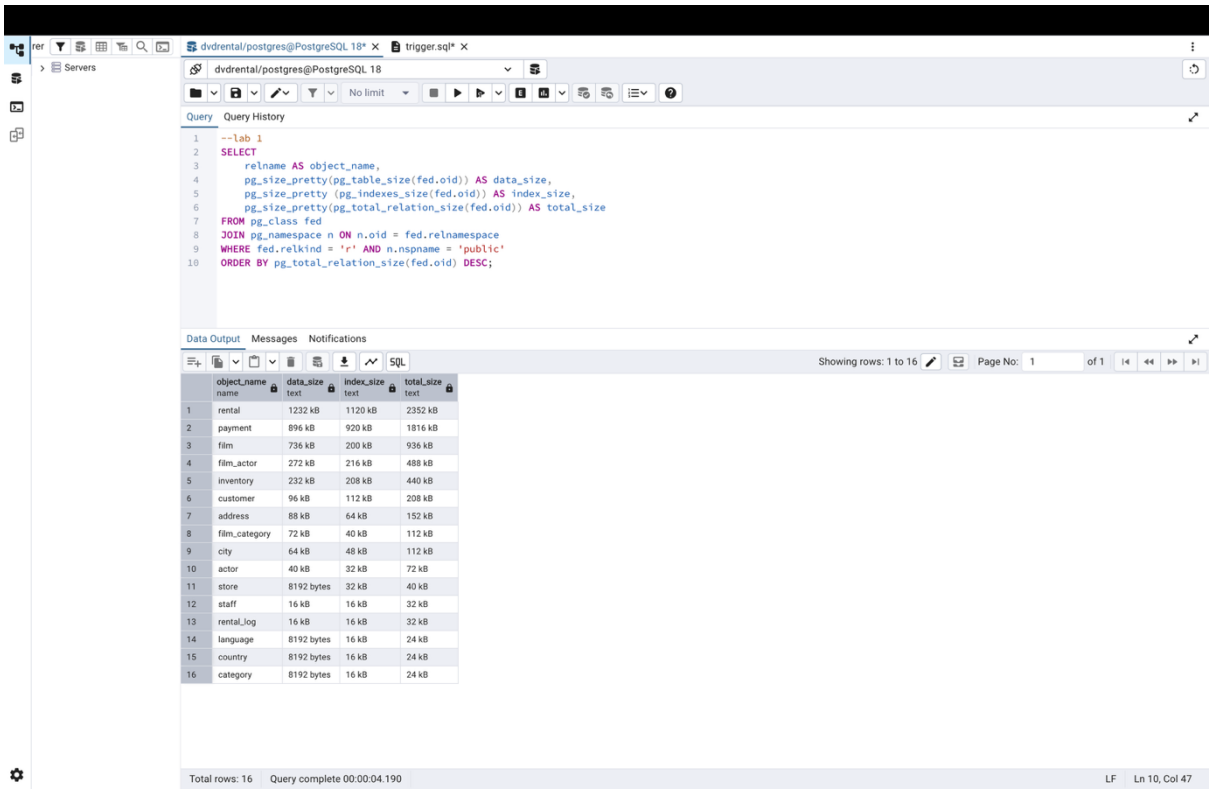
Lab 1:



The **payment** and **customer** tables are the most obvious "Index Heavy" tables dvdrental database because the storage dedicated to their indexes exceeds the storage for their actual data.

Lab 2:

```
12  --lab 2
13  SELECT pg_size_pretty(pg_indexes_size('payment'));
```

Data Output | Messages | Notifications

	pg_size_pretty text
1	920 kB

```
16  --Step 2: Create a heavy index on payment_date
17  CREATE INDEX idx_payment_date_long ON payment (payment_date, amount, customer_id);
```

Data Output | Messages | Notifications

```
CREATE INDEX

Query returned successfully in 5 secs 510 msec.
```

```
19  --Step 3: Check size again
20  SELECT pg_size_pretty ( pg_indexes_size ('payment ' ) ) ;
```

Data Output | Messages | Notifications

	pg_size_pretty text
1	1488 kB

- The initial index size was **920 kB**.
- After creating the new index, the size became **1488 kB**.
- The new index consumed **568 kB** of disk space (1488 - 920).
- Since each page in PostgreSQL is exactly 8 KB, the new index consumed **71 pages** (568 / 8).

Lab 3:

```sql
--lab 3
-- Create Partial Index
CREATE INDEX idx_expensive_films
ON film (replacement_cost)
WHERE replacement_cost > 25.00;
```

Data Output | Messages | Notifications

```
CREATE INDEX

Query returned successfully in 5 secs 120 msec.
```

```sql
-- Find the size of this specific index
SELECT relname, pg_size_pretty(pg_relation_size(relid))
FROM pg_stat_user_indexes
WHERE relname = 'idx_expensive_films';
```

Data Output | Messages | Notifications

	relname name	pg_size_pretty text

```sql
SELECT pg_size_pretty(pg_relation_size('idx_expensive_films'));
```

Data Output | Messages | Notifications

	pg_size_pretty text
1	16 kB

```sql
CREATE INDEX idx_full_replacement_cost ON film (replacement_cost);
```

Data Output | Messages | Notifications

```
CREATE INDEX

Query returned successfully in 9 secs 40 msec.
```

```
37  SELECT pg_size_pretty(pg_relation_size('idx_full_replacement_cost'));
```

Data Output | Messages | Notifications

	pg_size_pretty text
1	40 kB

- **Partial Index Size** (idx_expensive_films): **16 kB**.
- **Full Index Size** (idx_full_replacement_cost): **40 kB**.

Conclusion: Using a Partial Index reduced the storage requirement from 40 kB down to 16 kB, saving 60% of the disk space. This practically demonstrates the core benefits of a Partial Index. By using a WHERE clause to filter exactly which rows get indexed, you achieve huge storage savings. Furthermore, querying this index is highly efficient because a smaller tree depth results in faster traversal.